26. Veri Modelleme & ER Diyagramlarını AI ile Tasarlamak

1:1, 1:N ve N:M İlişkilerini Doğru Kurarak Veri Tutarlılığını Sağlamak
Bölüm 5 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Birçok junior mühendis N:M (Çoka-Çok) ilişkileri kurarken neden ara tabloyu (Junction Table) unutur ve AI ile kusursuz ER diyagramı nasıl çizdirilir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Veri modelleme, kod yazmadan önce gerçek dünyanın nesnelerini ve kurallarını tablolara aktarma sanatıdır. - 📌 1:N (Bire-Çok) ilişkide yabancı anahtar (Foreign Key) daima 'Çok' olan tarafa konur. - 📌 N:M (Çoka-Çok) ilişkiler doğrudan kurulamaz; arada iki foreign key tutan bir ara tablo (Junction Table) zorunludur.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir okul hayal edin: - Her öğrencinin tek bir kimlik kartı vardır: **Bire-Bir (1:1)** - Bir öğretmenin onlarca öğrencisi olabilir: **Bire-Çok (1:N)** - Peki öğrenciler ve dersler? Bir öğrenci 5 farklı derse girer, bir derste 30 farklı öğrenci vardır! İşte bu **Çoka-Çok (N:M)** ilişkidir!

Eğer öğrenci tablosunun içine dersin adını yazarsanız, öğrenci 5 ders aldığında öğrencinin adını 5 kez tekrar yazmak zorunda kalırsınız!

Çözüm: Ortaya bir sınıf yoklama defteri koymaktır (Junction Table). Defterde sadece: *"Öğrenci 101, Ders 201'e kayıtlıdır"* yazar.

Veri tabanı tasarımı, bu ilişkileri mantıklı defterlere bölme sanatıdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

İlişki Türleri ve Yabancı Anahtar (Foreign Key) Kuralları:

```
1. BİRE-BİR (1:1):
   [Kullanıcı (User)] <——> [Kullanıcı Profili (Profile)]
   * Foreign Key profil tablosuna konur ve UNIQUE yapılır.

2. BİRE-ÇOK (1:N):
   [Müşteri (Customer)] 1 <—— N [Siparişler (Orders)]
   * Foreign Key daima 'Çok' tarafına (Orders tablosuna customer_id) konur.

3. ÇOKA-ÇOK (N:M):
   [Öğrenciler] N <—— [Kayıtlar (Enrollments)] ——> M [Dersler]
   * Ara tablo (Junction Table) her iki tablonun ID'sini tutar.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Veritabanı Mimarı ve ER Uzmanı
Bir 'Online Kurs ve Eğitmen Platformu' (Udemy benzeri) için veritabanı şeması
tasarlayacağız.

## GEREKSİNİMLER:
- Bir eğitmenin birden fazla kursu olabilir (1:N).
- Bir kursun birden fazla kategorisi ve bir kategorinin birden fazla kursu olabilir
(N:M).
- Kullanıcılar kurslara kaydolabilmelidir (N:M).
- Her kursun video dersleri (Lessons) sırayla listelenmelidir (1:N).

## GÖREV:
1. Bu sistemin Mermaid ERD (Entity Relationship Diagram) şemasını çiz.
2. Prisma ORM formatında `schema.prisma` dosyasını foreign key ve index'leriyle
eksiksiz yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü Tasarım: Çoka-Çok ilişkiyi string array olarak saklamak
model Course {
  id          String   @id @default(uuid())
  title       String
  categories  String[] // HATA: Arama yapılamaz, kategori adı değişirse tüm kurslar
bozulur!
  studentIds  String[] // HATA: 50.000 öğrenci kaydolduğunda döküman patlar!
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Profesyonel Prisma Şeması: İlişkisel Bütünlük ve İndeksleme
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id           String       @id @default(uuid())
  email        String       @unique
  fullName     String
  taughtCourses Course[]    @relation("InstructorCourses")
  enrollments  Enrollment[]
  createdAt    DateTime     @default(now())
}

model Course {
  id           String       @id @default(uuid())
  title        String
  instructorId String
  instructor   User         @relation("InstructorCourses", fields: [instructorId],
references: [id])
  lessons      Lesson[]
  enrollments  Enrollment[]
  categories   CourseCategory[]
  createdAt    DateTime     @default(now())

  @@index([instructorId])
}

model Lesson {
  id        String   @id @default(uuid())
  courseId  String
  course    Course   @relation(fields: [courseId], references: [id], onDelete:
Cascade)
  title     String
  order     Int
  videoUrl  String

  @@index([courseId, order])
}

// N:M İlişkisi İçin Ara Tablo (Junction Table)
model Enrollment {
  userId     String
  courseId   String
  user       User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  course     Course   @relation(fields: [courseId], references: [id], onDelete:
Cascade)
  enrolledAt DateTime @default(now())

  @@id([userId, courseId]) // Bir kullanıcı aynı kursa iki kez kaydolamaz!
}
```

💡 **Kod Analizi:** `@@id([userId, courseId])` bileşik anahtarı (compound key) sayesinde bir öğrencinin aynı kursa mükerrer kaydolması veritabanı seviyesinde imkansız kılınmıştır. `onDelete: Cascade` ile kurs silindiğinde derslerinin öksüz (orphaned) kalması engellenir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Çoka-çok ilişkilerde ID listesini JSON veya virgülle ayrılmış string olarak tek bir kolona tıkıştırmak.
  - **Çözüm:** Her zaman iki foreign key içeren standart bir ara tablo (Junction Table) oluşturun.
- **Tuzak:** Foreign Key alanlarına indeks koymayı unutmak.
  - **Çözüm:** Prisma'da `@@index([foreignKey])` veya SQL'de `CREATE INDEX` ile arama hızını garanti edin.
- **Tuzak:** Silme (Delete) kurallarını belirlememek ve veritabanında öksüz (orphan) kayıtlar bırakmak.
  - **Çözüm:** İş mantığına göre `ON DELETE CASCADE` veya `ON DELETE RESTRICT` kurallarını açıkça tanımlayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 26: Veri Modelleme - ER Diyagramları ve Prisma Şeması*

- 🖥️ **Slayt 1:** Veri modelleme nedir? Gerçek dünyayı tablolara dökmek.
- 🖥️ **Slayt 2:** 1:1, 1:N ve N:M ilişkilerinin matematiksel kuralları.
- 🖥️ **Slayt 3:** Junction Table (Ara Tablo) ve Compound Primary Key kullanımı.
- 🖥️ **Slayt 4:** Prisma ORM ile 5 dakikada kurumsal şema oluşturmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Mermaid.js ile bir e-ticaret ER diyagramı çizin. Ardından AI asistanına bu diyagramı verip hatasız bir `schema.prisma` üretmesini sağlayın ve `npx prisma format` ile doğrulayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Kendi projeniz için Mermaid.js formatında en az 4 tablodan oluşan bir ER diyagramı çizin ve markdown önizlemesinde inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: İlişkisel veritabanlarında Çoka-Çok (Many-to-Many) bir ilişkiyi modellemenin standart yolu nedir? - A) Tüm ID'leri virgülle ayırıp tek bir hücreye yazmak - B) Her iki tablonun birincil anahtarlarını (Primary Key) yabancı anahtar (Foreign Key) olarak tutan bir ara tablo (Junction Table) oluşturmak - C) Veritabanını silip baştan kurmak - D) Sadece NoSQL kullanmak > **Doğru Yanıt:** B > **Açıklama:** N:M ilişkiler, her iki tarafın ilişkisini satır bazında tutan bir ara tablo (Junction Table) ile iki adet 1:N ilişkiye dönüştürülerek modellenir.

Soru 2: Bir tabloda `ON DELETE CASCADE` kuralı tanımlandığında ne gerçekleşir? - A) Bilgisayar yeniden başlar - B) Ana tablodaki kayıt silindiğinde, ona bağlı olan tüm alt tablodaki ilişkili kayıtlar da otomatik olarak güvenle silinir - C) Tablo şifrelenir - D) Silme işlemi tamamen yasaklanır > **Doğru Yanıt:** B > **Açıklama:** CASCADE silme kuralı, ebeveyn kayıt silindiğinde geride öksüz (orphan) kayıtlar kalmasını önlemek için çocuk kayıtları da siler.